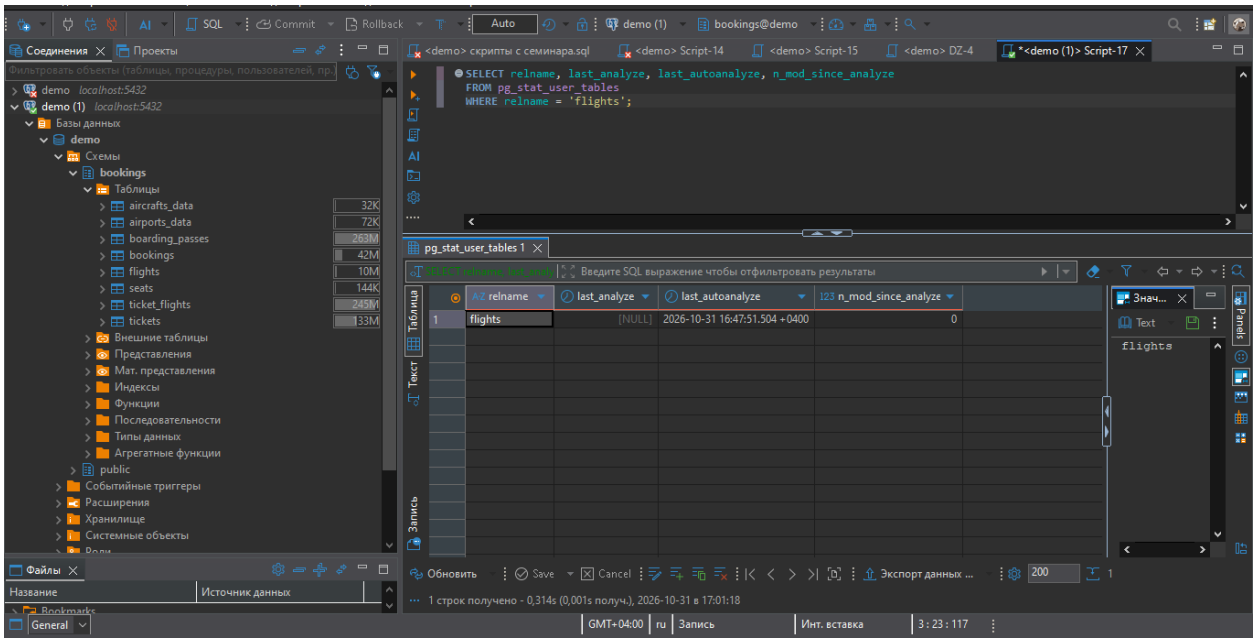
7 ДЗ

Яицкая Ангелина

ДЭ 16-25

Задание 1

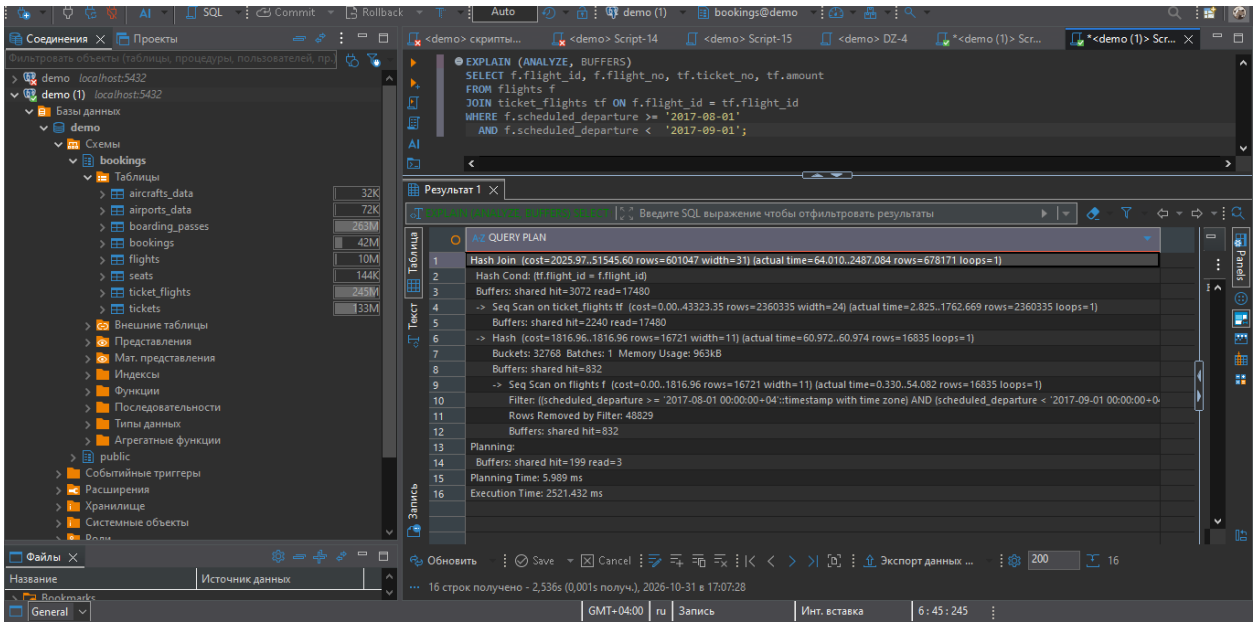


Дата последнего автоанализа: 6:47:51 +0400. Количество модификаций с момента анализа: 0. Статистика актуальна (0 < 10% от общего размера таблицы), поэтому ANALYZE flights не требуется.

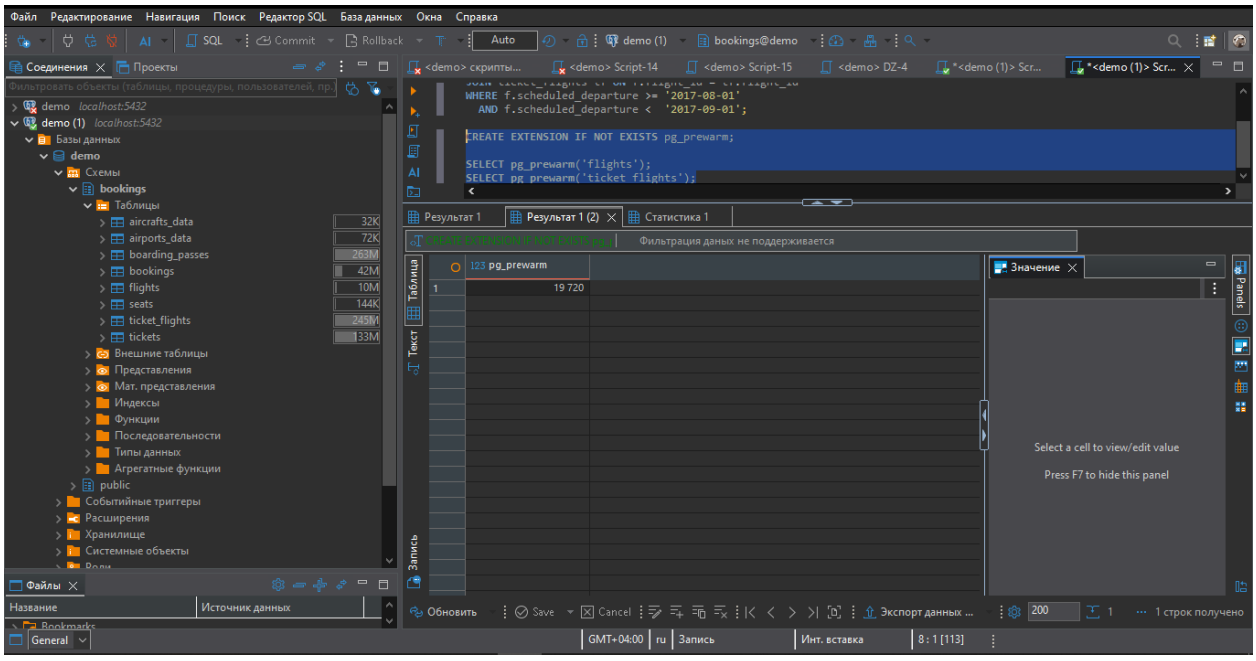
Признак устаревания: поле n_mod_since_analyze показывает число изменений с момента последнего анализа. Если это число превышает 10% от общего числа строк (count(*)), статистика устарела и её нужно обновить вручную через ANALYZE.

Задание 2

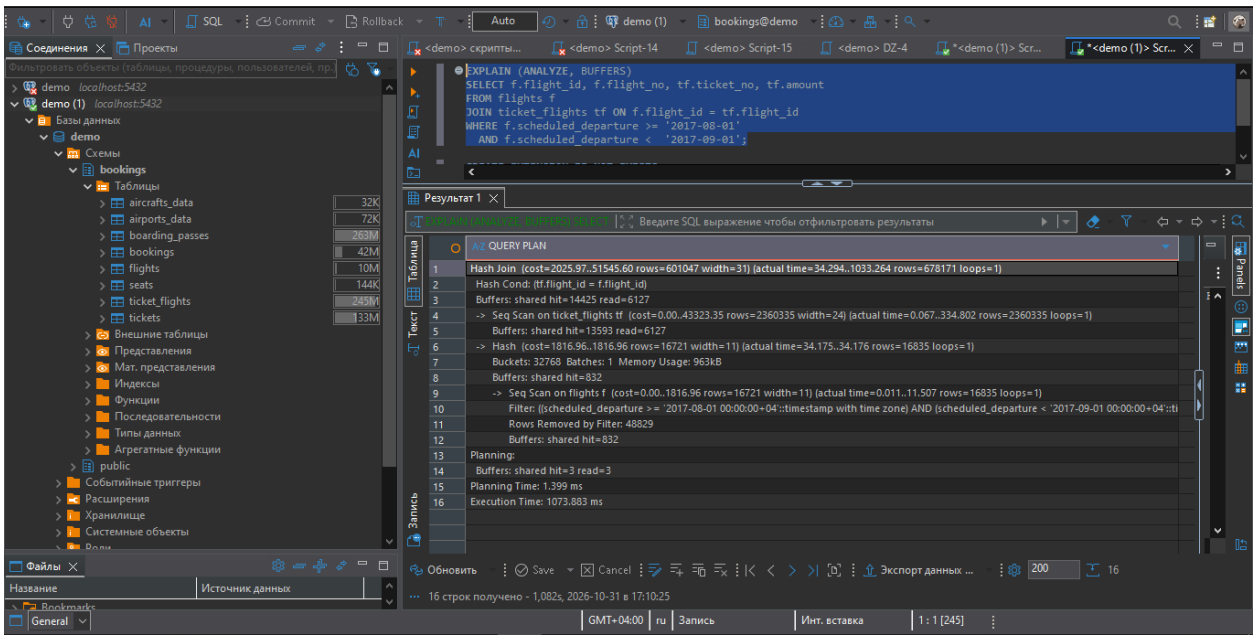
До прогрева:



прогрев



После прогрева



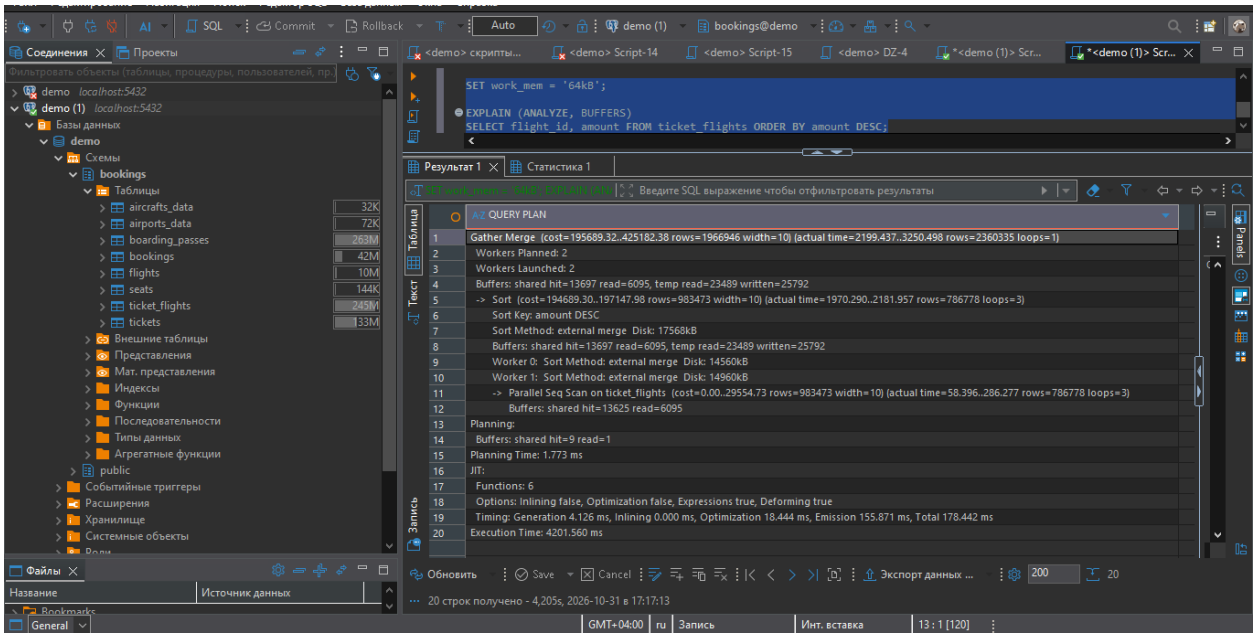
До прогрева: Запрос выполнился за 2521 мс. В плане видно shared read=17480 и shared hit=30. Это означает, что СУБД пришлось читать данные с диска, так как кэш (shared_buffers) был холодным.

После прогрева: Я создала расширение pg_prewarm и прогрела обе таблицы (SELECT pg_prewarm('flights'); SELECT pg_prewarm('ticket_flights');). Повторный EXPLAIN показал shared read=6127 и shared hit=14425. Время выполнения сократилось до 1073 мс (в 2.3 раза быстрее).

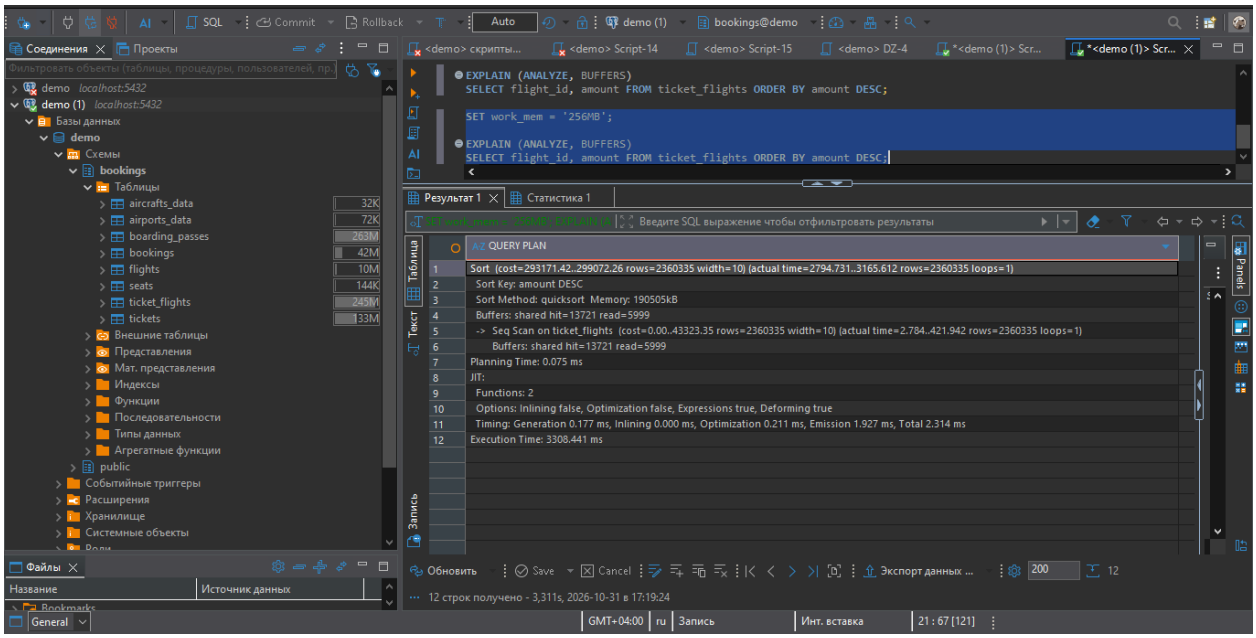
Вывод: pg_prewarm принудительно загружает страницы таблиц в оперативную память (в shared_buffers). Это избавляет СУБД от необходимости обращаться к диску (I/O) при выполнении запроса, что значительно снижает время выполнения. На скриншотах видно, как метрика read заменяется на hit.

Задание 3

work_mem = '64kB'



work_mem = '256MB'



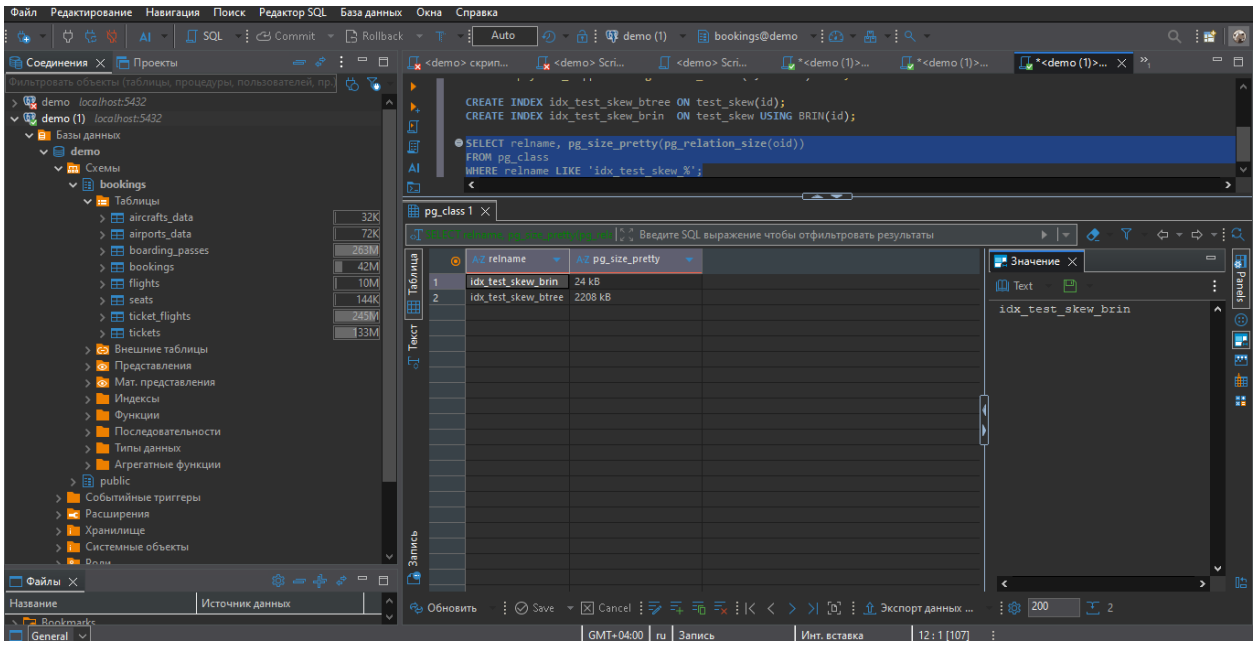
При work_mem = '64kB' сортировка не помещается в выделенную память. В плане появляется Sort Method: external merge Disk: 17568kB, а также метрики temp read и temp written. Это означает, что PostgreSQL сбрасывает промежуточные данные сортировки во временные файлы на диске. Время выполнения

–4201 мс.

После увеличения work_mem до '256MB' сортировка выполняется целиком в памяти: Sort Method: quicksort Memory: 190505kB. Временные файлы на диске больше не используются, время выполнения сокращается до 3308 мс.

Почему возникает чтение с диска, если таблица в shared_buffers? shared_buffers хранит страницы самой таблицы, но для операции ORDER BY PostgreSQL нужно выделить дополнительную память под сортировку (буферы сортировки). Если этой памяти (work_mem) не хватает, СУБД вынуждена создавать временные файлы на диске, чтобы сохранить промежуточные результаты. Эти временные файлы не являются частью таблицы и не находятся в shared_buffers.

Задание 4



Создана таблица test_skew на 100 000 строк. Созданы два индекса по колонке id: B-Tree и BRIN.

Результат сравнения размеров:

idx_test_skew_btree –2208 kB.

idx_test_skew_brin –24 kB.

Почему BRIN занимает меньше места: B-Tree хранит указатель на каждую строку таблицы (по сути, копию ключа + ссылку на страницу), поэтому его размер растёт линейно с количеством строк. BRIN (Block Range Index) хранит только минимальное и максимальное значение для группы страниц (по умолчанию –для 128 страниц). Он не знает точного местоположения каждой строки, а лишь определяет диапазон. Поэтому BRIN в десятки раз компактнее.

Когда использовать:

B-Tree –для OLTP-поиска по ID (WHERE id = 5), для точечных запросов и проверок уникальности. Он даёт точный результат быстро.

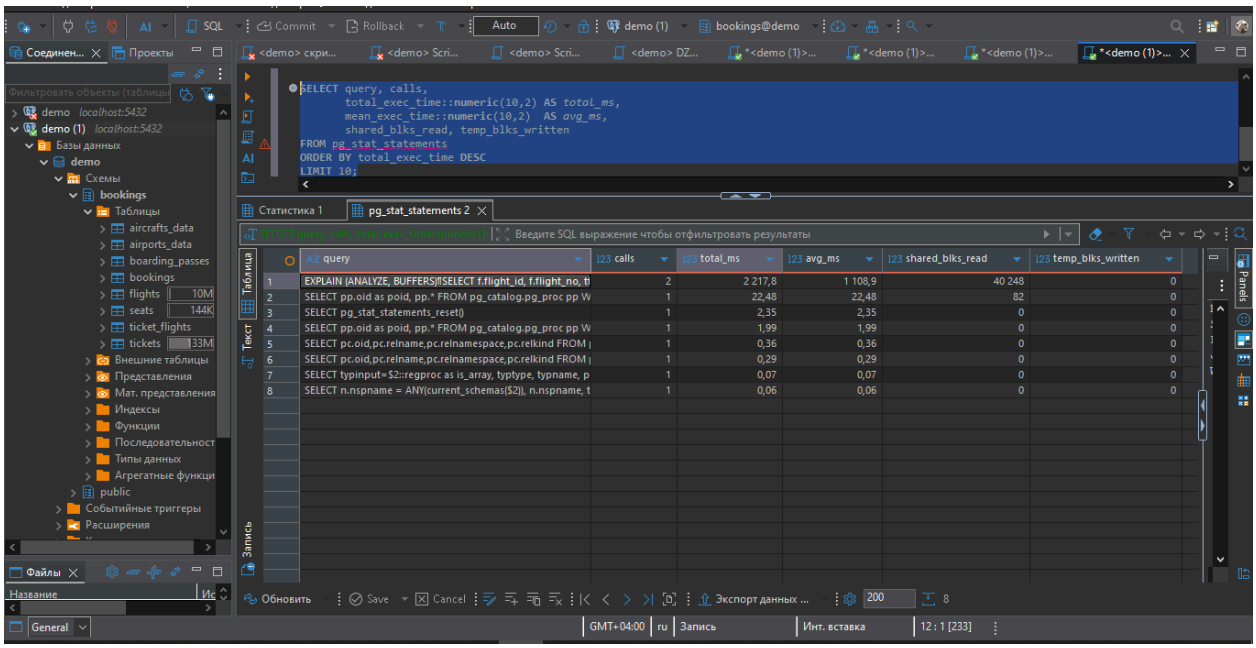
BRIN –для больших таблиц с упорядоченными данными (например, логи с полем timestamp, или наш id, который растёт монотонно). Идеально для запросов по диапазону (WHERE id BETWEEN 1000 AND 2000).

Как помогли бы fillfactor и партиционирование:

fillfactor (например, 70%) оставляет свободное место на страницах индекса. Это снижает количество page splits при вставках в середину (характерно для случайных ключей) и уменьшает фрагментацию.

Партиционирование разбивает большую таблицу на части по какому-то признаку (например, по диапазону дат). Это уменьшает размер каждого индекса, ускоряет поиск (сканируется только нужная партиция) и упрощает обслуживание (VACUUM, ANALYZE для одной партиции).

Задание 5



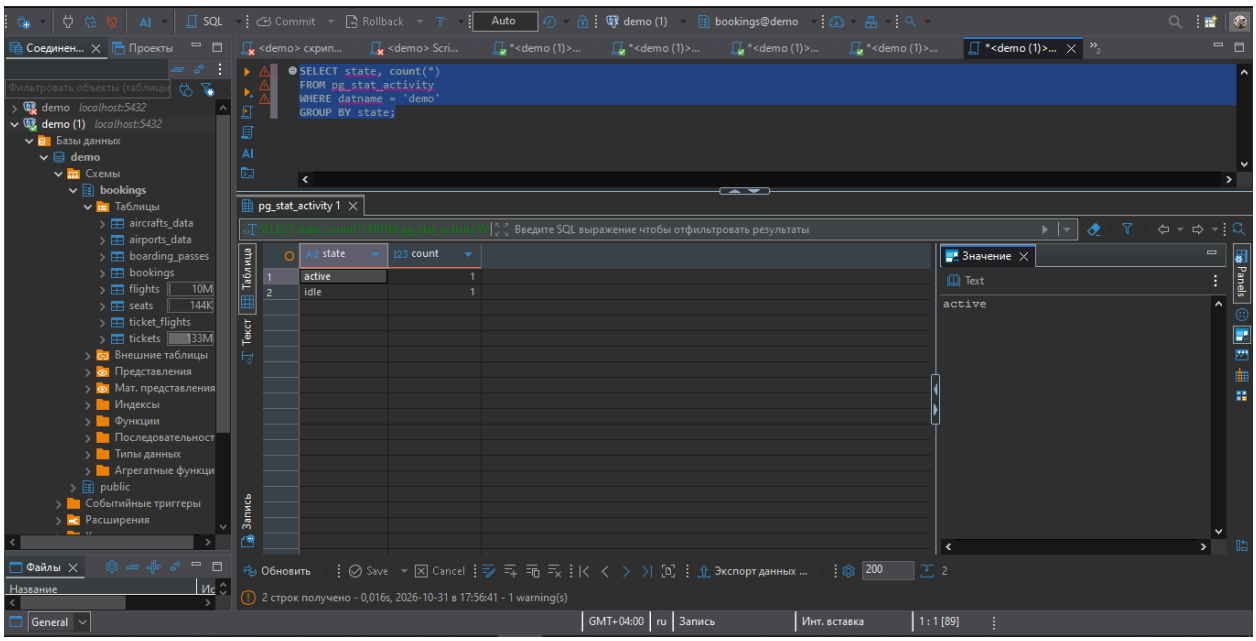
Вывод: В топе самых медленных запросов оказался аналитический запрос с JOIN таблиц flights и ticket_flights. Он вызывался 2 раза, суммарное время выполнения –2217.8 мс, среднее –1108.9 мс.

Метрика shared_blks_read = 40248 показывает, что при выполнении было прочитано с диска (или из ОС) около 40 тысяч страниц (8 КБ каждая), что говорит о холодном кэше или нехватке shared_buffers для таких объемов. Метрика temp_blks_written = 0 означает, что временные файлы на диск не сбрасывались (хватило оперативной памяти).

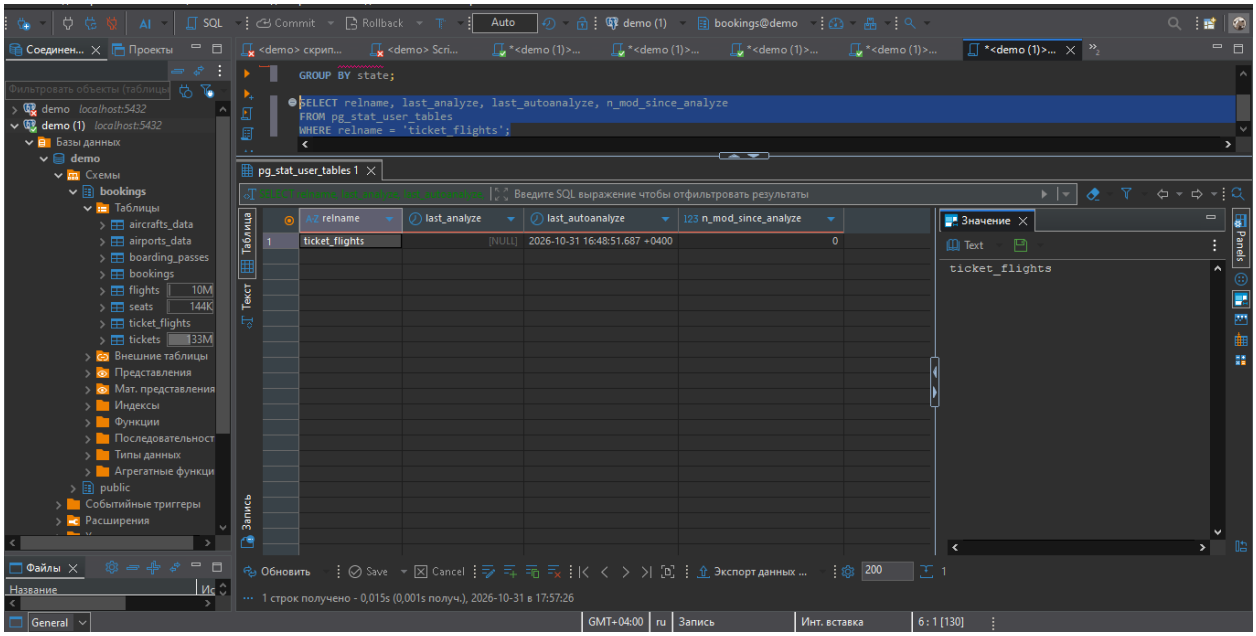
Эффективность: Запрос довольно тяжелый, так как выполняется последовательное сканирование больших таблиц (Seq Scan) и хеш-соединение (Hash Join). Для ускорения можно было бы добавить индексы по полям фильтрации (scheduled_departure) и соединения (flight_id).

6. Невыполненные задачи с практики

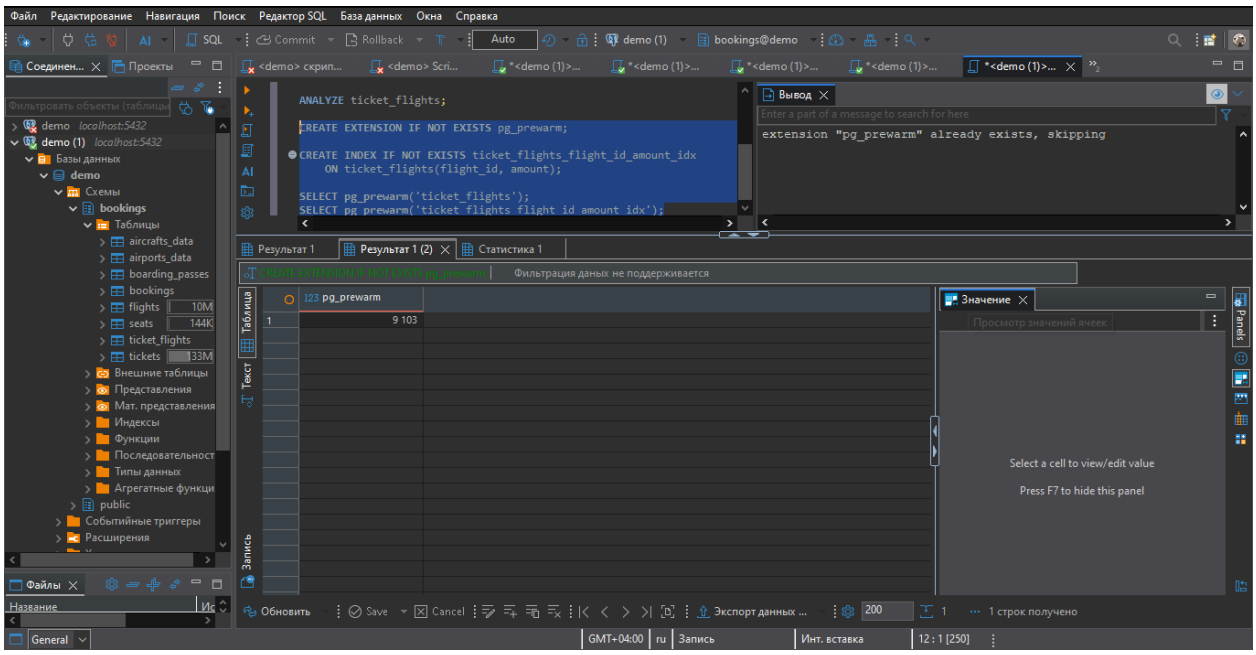
Раздел 3. **Практикум 1: Pre-flight Checklist**



Запрос к pg_stat_activity показывает, что в базе demo активна 1 сессия (моя), остальные находятся в состоянии idle. Это важно, потому что перед замерами мы должны убедиться, что нет фоновых тяжелых запросов от других пользователей, которые могли бы исказить результаты.

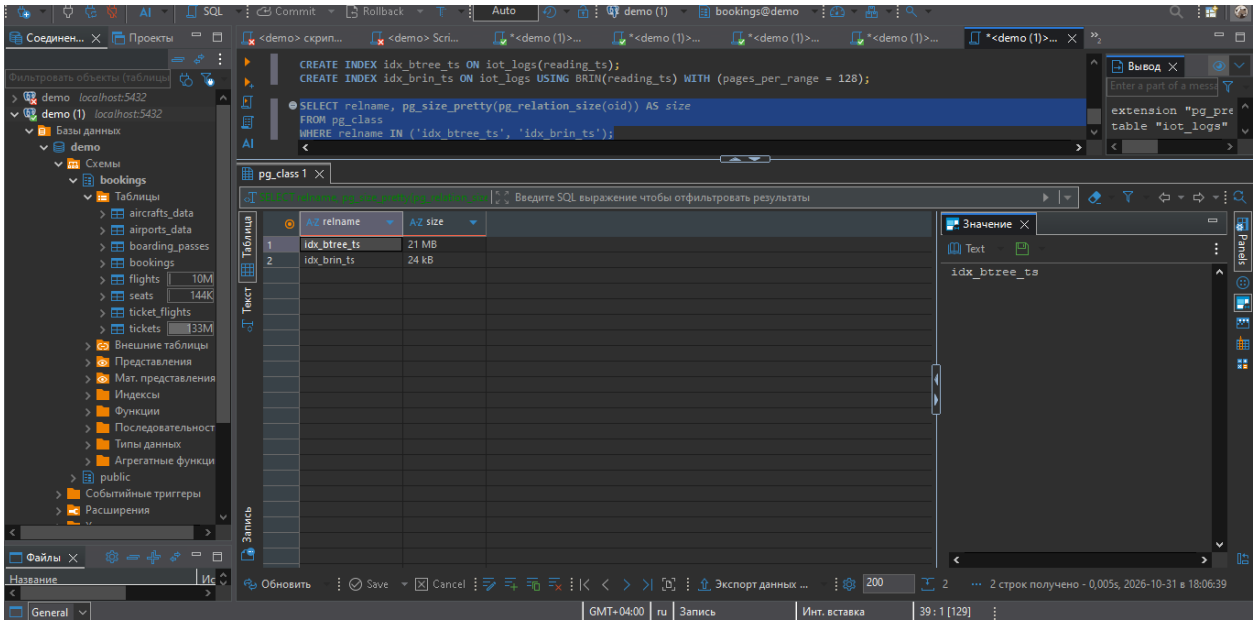


Запрос к pg_stat_user_tables показывает, что автоанализ таблицы ticket_flights прошел недавно (last_autoanalyze = 16:48:51), а количество модификаций с тех пор равно 0 (n_mod_since_analyze). Статистика актуальна, дополнительный ANALYZE не требуется

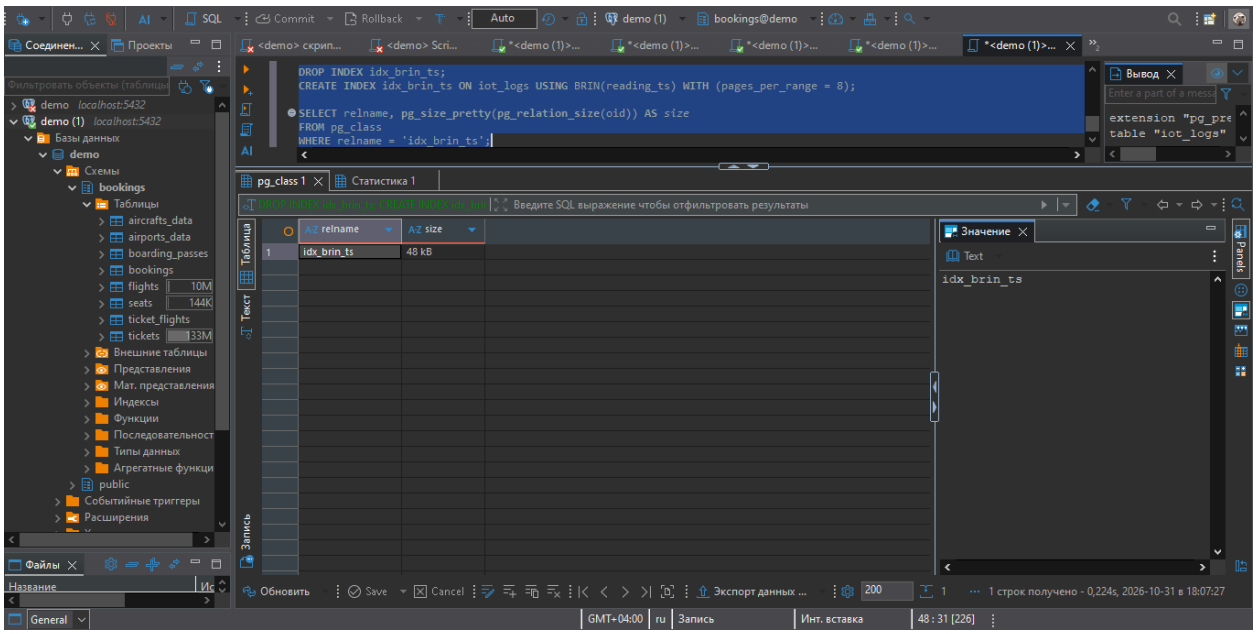


Я создала индекс ticket_flights_flight_id_amount_idx и выполнила pg_prewarm.
Результат 9103 означает, что в оперативную память (shared_buffers) было загружено 9103 страницы индекса. Это обеспечит горячий кэш для дальнейших замеров.

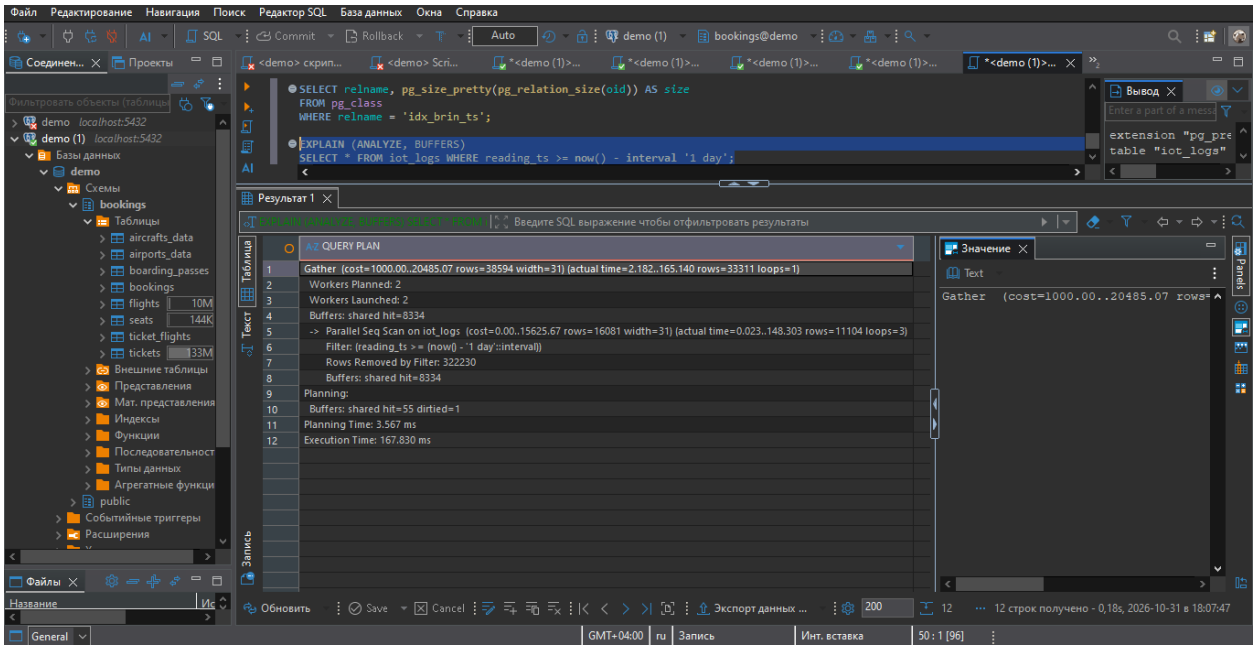
Практикум 2: Перекос B-Tree, BRIN vs B-Tree



B-Tree индекс на миллион строк занимает 21 MB. BRIN-индекс с pages_per_range = 128 занимает всего 24 kB –почти в 1000 раз меньше.



После пересоздания BRIN-индекса с pages_per_range = 8 его размер увеличился до 48 kB (в 2 раза больше).



План запроса на выборку за последний день показывает Parallel Seq Scan, а не Index Scan. Это связано с тем, что данные в колонке reading_ts генерировались случайно (now() - random() * 30 days), то есть физический порядок строк не соответствует порядку reading_ts. Для BRIN-индекса это критично: он эффективен только тогда, когда данные физически упорядочены (например, логи пишутся по времени). Из-за отсутствия корреляции планировщик выбрал последовательное сканирование.

Влияние pages_per_range:

128 (больше): Меньше записей в индексе, индекс компактнее (24 kB). Но каждая запись покрывает больше страниц, поэтому точность фильтрации ниже (в выборку попадает больше «лишних» страниц).

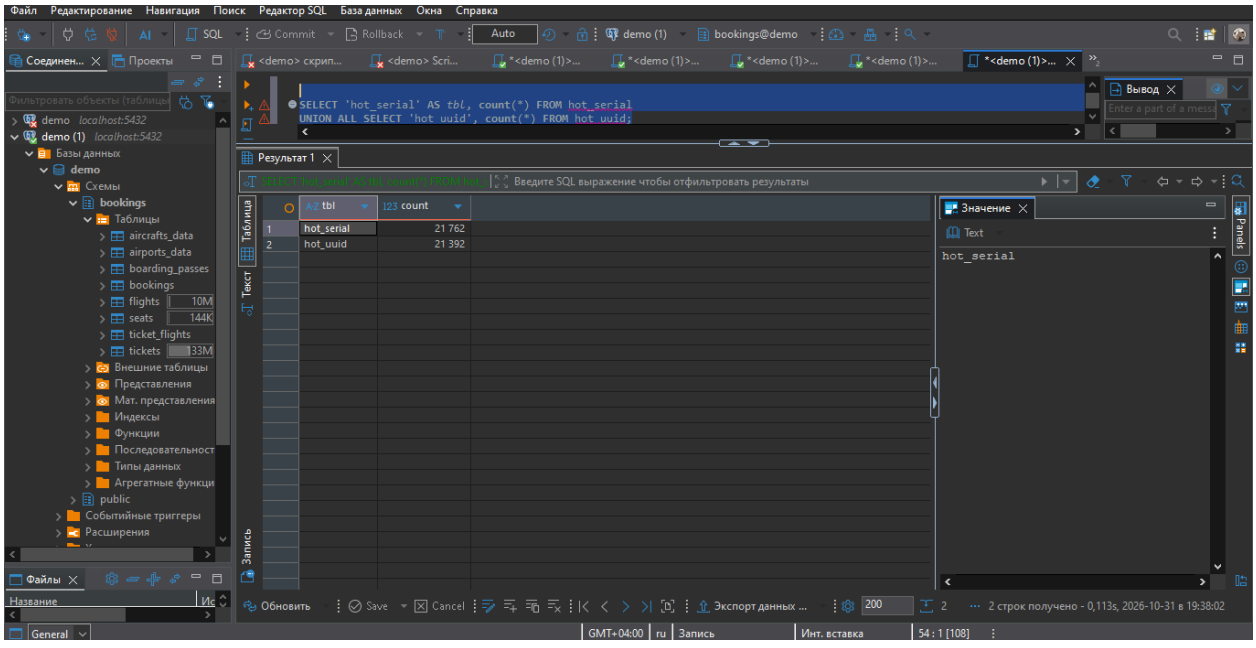
8 (меньше): Записей больше, индекс крупнее (48 kB). Но точность выше –база может отсеять больше ненужных страниц.

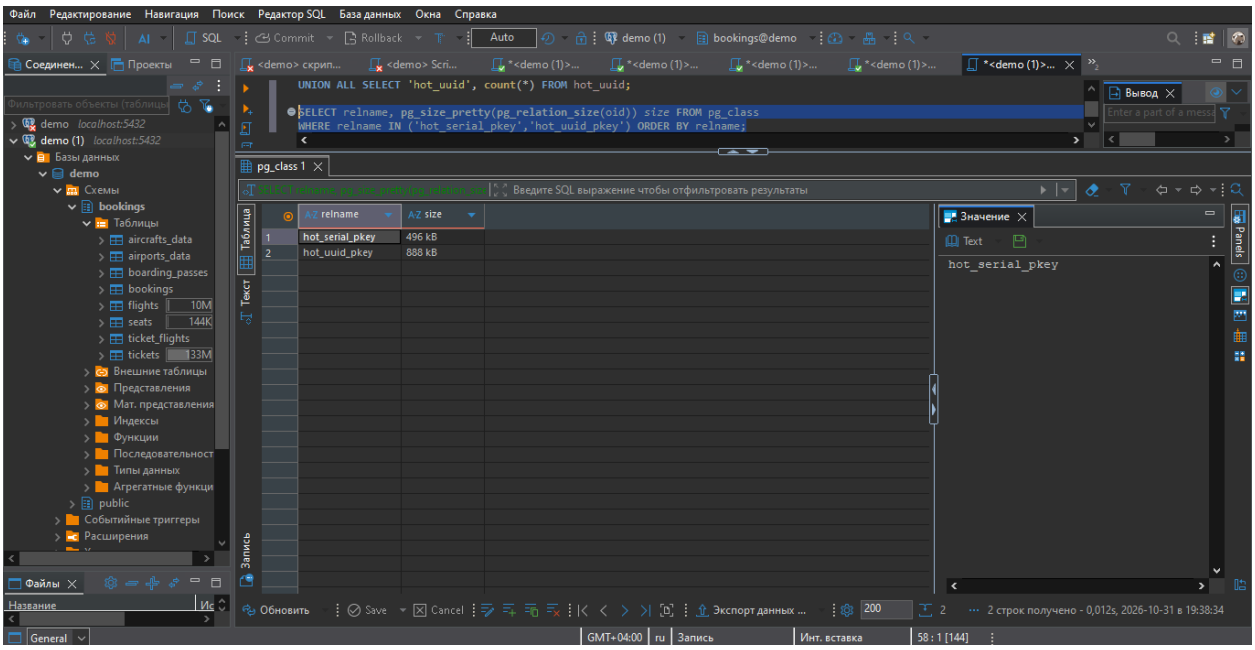
Вывод: BRIN идеален для больших таблиц с естественной физической сортировкой (логи, time-series). B-Tree лучше для OLTP (точечный поиск по ID).

Практикум 4: Latch Contention

```
$ bash seminar7_bench.sh
NOTICE:  table "hot_serial" does not exist, skipping
NOTICE:  table "hot_uuid" does not exist, skipping
DROP TABLE
CREATE TABLE
CREATE TABLE
=== serial ===
number of transactions actually processed: 21762
tps = 1089.143563 (without initial connection time)
     50  LWLock|WALWrite
      8  IO|WALSync
      6  |
      3  Client|ClientRead
      2  IO|WALWrite
=== uuid ===
number of transactions actually processed: 21392
tps = 1070.604870 (without initial connection time)
     64  LWLock|WALWrite
     10  IO|WALSync
      2  |
      2  Client|ClientRead
   tbl      | count
------------+-------
 hot_serial | 21762
 hot_uuid   | 21392
(2 rows)

     relname     |  size
-----------------+--------
 hot_serial_pkey | 496 kB
 hot_uuid_pkey   | 888 kB
(2 rows)
```





Для сравнения перекосов были созданы две таблицы:

hot_serial с первичным ключом BIGSERIAL.

hot_uuid с первичным ключом UUID DEFAULT gen_random_uuid().

С помощью утилиты pgbench (8 клиентов, 4 потока, 20 секунд) в обе таблицы выполнялась вставка строк.

Результаты:

hot_serial: 21762 транзакции, tps = 1089. В топе wait events: LWLock|WALWrite (50), IO|WALSync (8).

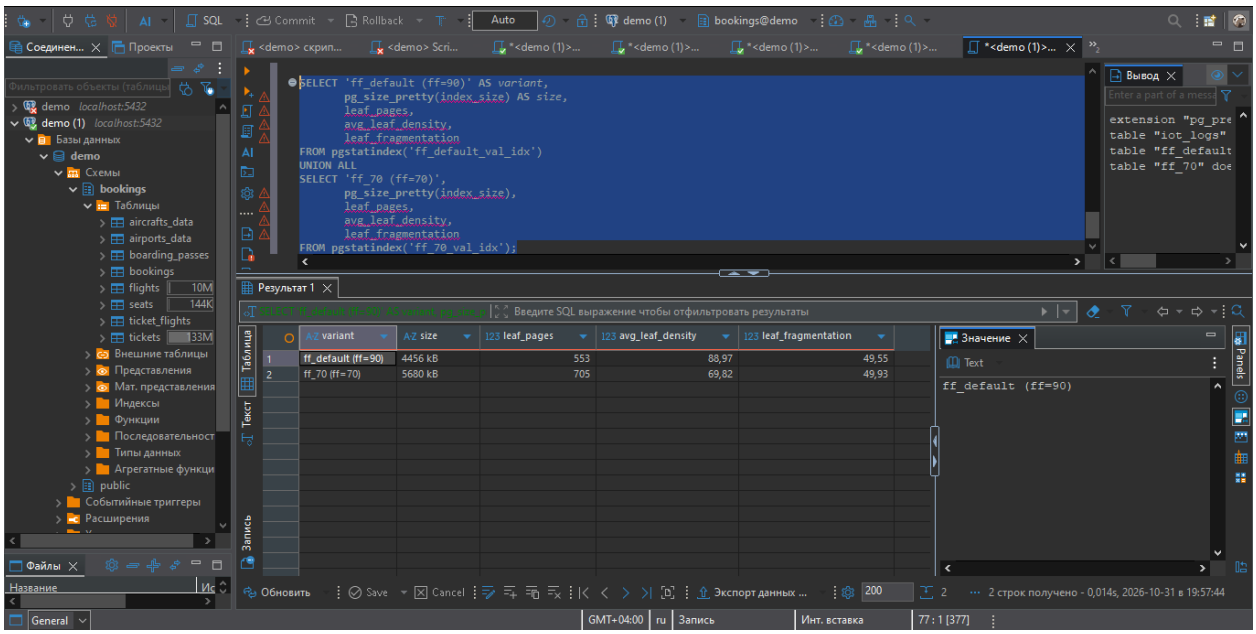
hot_uuid: 21392 транзакции, tps = 1070. В топе wait events: LWLock|WALWrite (64), IO|WALSync (10).

Размеры индексов: hot_serial_pkey = 496 kB, hot_uuid_pkey = 888 kB.

Вывод: В таблице hot_serial монотонные ключи (BIGSERIAL) создают hot spot (перекос) –все вставки идут в правый лист B-Tree. В hot_uuid ключи случайные, поэтому вставки распределяются по разным листьям. Однако из-за того, что UUIDv4 случайны, они вызывают больше page splits (раздробления страниц), из-за чего индекс hot_uuid_pkey занимает почти в 2 раза больше места (888 kB против 496 kB).

Что такое pgbench: Это утилита для нагрузочного тестирования PostgreSQL, встроенная в дистрибутив. Она имитирует работу множества клиентов, выполняющих SQL-запросы, и измеряет производительность (транзакции в секунду, задержки).

Практикум 5: fillfactor и Page Splits



Были созданы две таблицы ff_default и ff_70 по 200 000 строк (100 000 до создания индекса и 100 000 после). Для индекса ff_default_val_idx использовался fillfactor = 90 (по умолчанию),
для ff_70_val_idx –fillfactor = 70.

Результаты:

ff_default (ff=90): размер 4456 kB, leaf_pages = 553, avg_leaf_density = 88.97, leaf_fragmentation = 49.55.

ff_70 (ff=70): размер 5680 kB, leaf_pages = 705, avg_leaf_density = 69.82, leaf_fragmentation = 49.93.

Вывод:

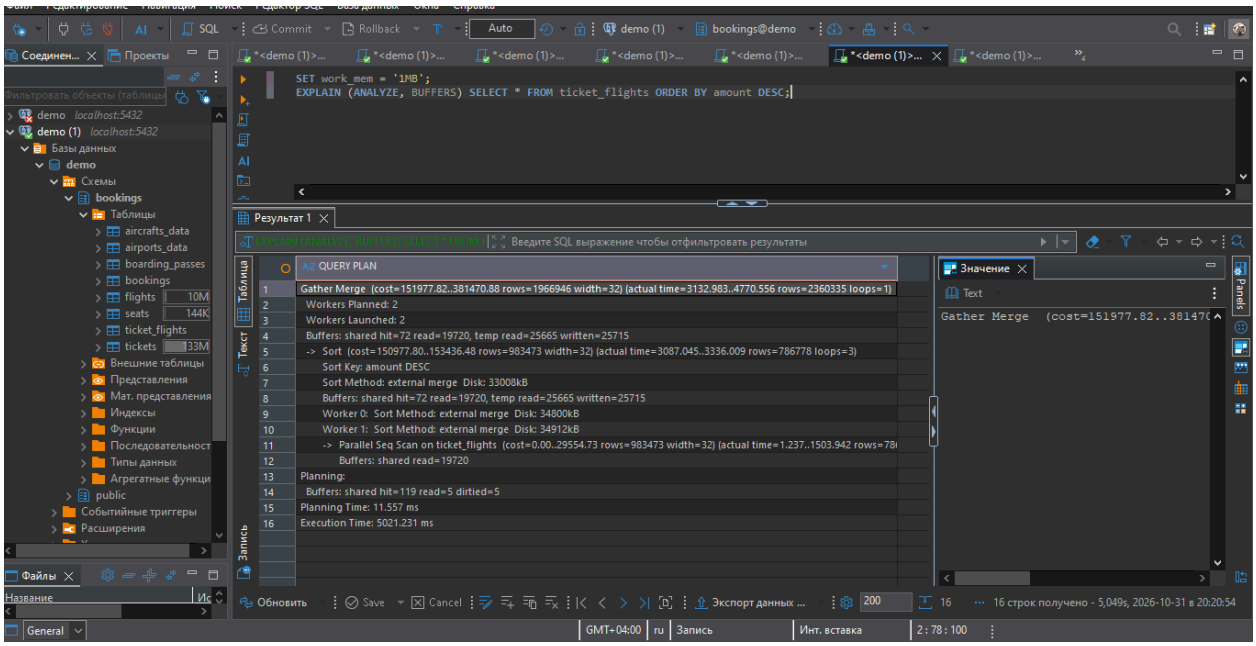
avg_leaf_density –это средняя заполненность листовых страниц индекса (в процентах). При fillfactor = 90 страницы заполнены на ~89%, при fillfactor = 70 –на ~70%.

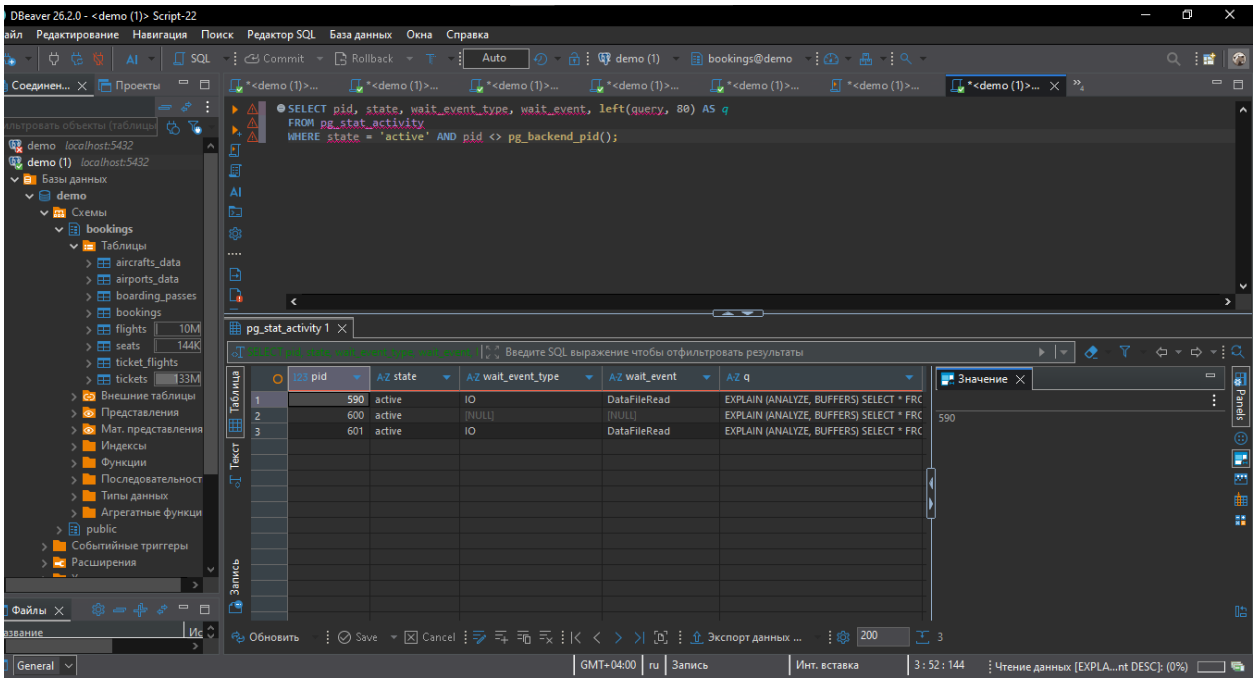
leaf_fragmentation –процент «мусора» (фрагментации) на страницах. В обоих случаях он около 50%, но это ожидаемо, так как после создания индекса было вставлено ещё 100 000 случайных значений, вызвавших page splits (раздробление страниц).

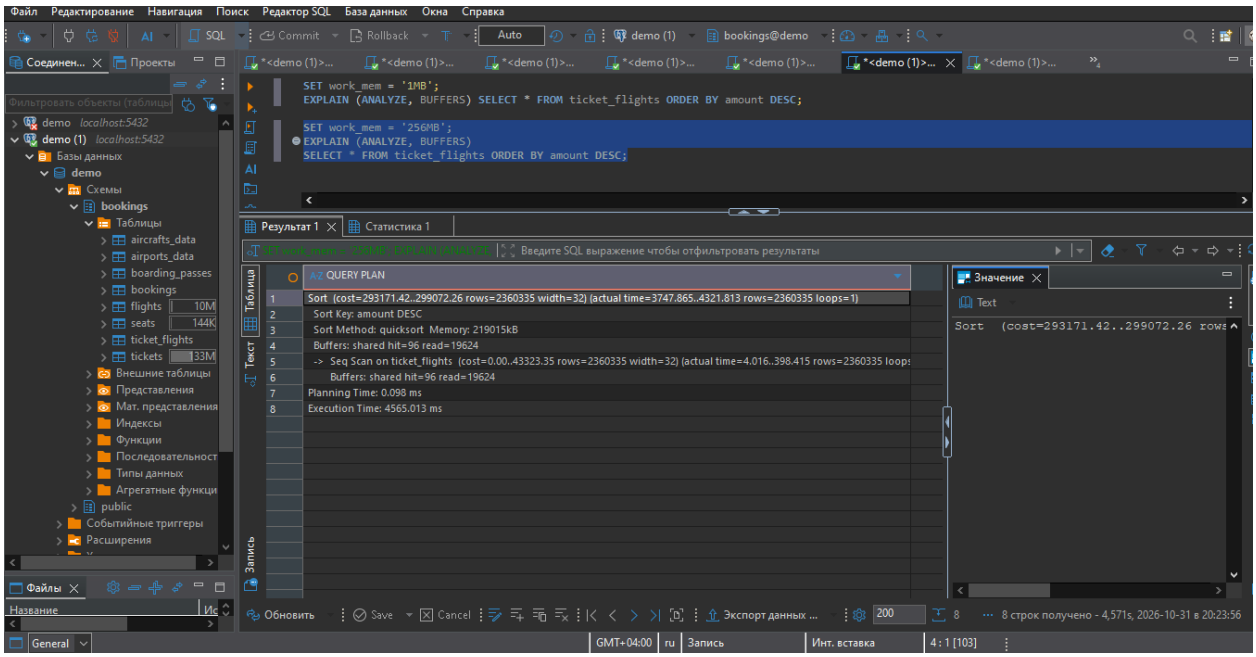
Индекс с fillfactor = 70 больше по размеру (5680 kB), потому что каждая страница заполнена меньше, и для хранения того же количества строк требуется больше страниц. Однако он лучше подходит для нагрузки на запись (много INSERT/UPDATE): за счёт свободного места на страницах уменьшается количество page splits.

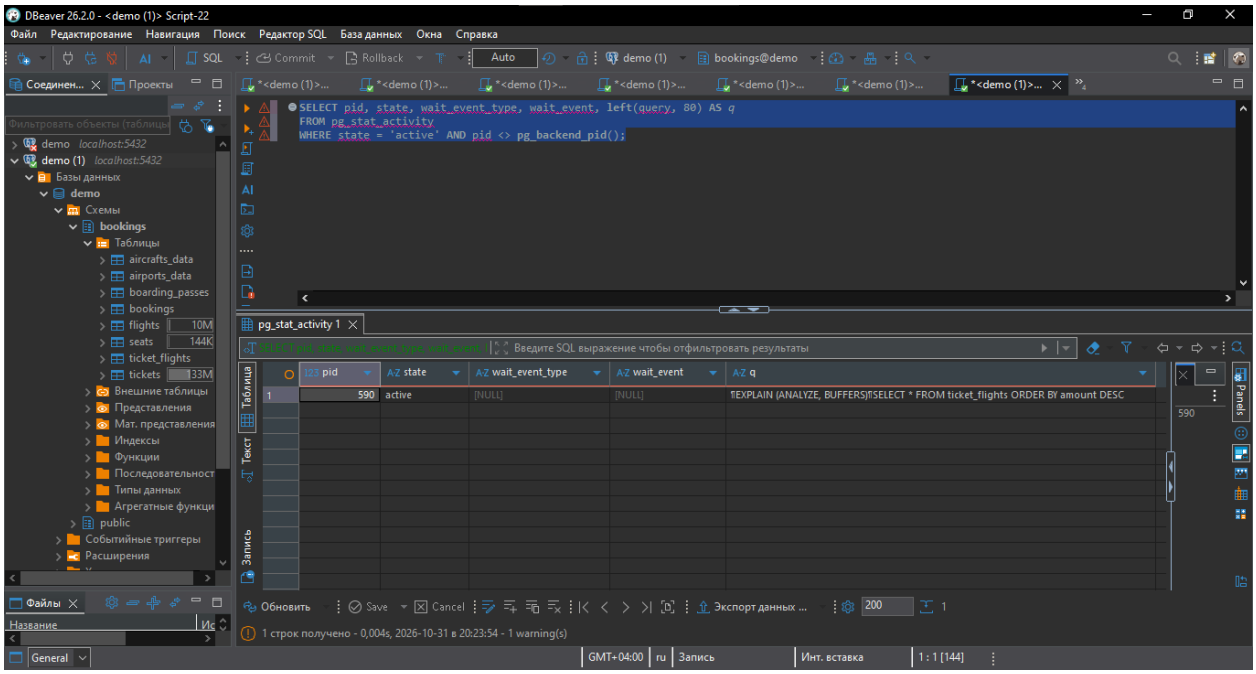
fillfactor ниже 90 применяют, когда таблица или индекс часто обновляются (особенно в случайные места). Если данные только читаются, fillfactor = 100 (или 90) даёт более компактный индекс.

Практикум 6: Wait Events Forensics


SET work_mem = '1MB';
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM ticket_flights ORDER BY amount DESC;
Результат 1
QUERY PLAN
Gather Merge (cost=151977.82..381470.88 rows=1966946 width=32) (actual time=3132.983..4770.556 rows=2360335 loops=1)
Workers Planned: 2
Workers Launched: 2
Buffers: shared hit=72 read=19720, temp read=25665 written=25715
-> Sort (cost=150977.80..153436.48 rows=983473 width=32) (actual time=3087.045..3336.009 rows=786778 loops=3)
Sort Key: amount DESC
Sort Method: external merge Disk: 33008kB
Buffers: shared hit=72 read=19720, temp read=25665 written=25715
Worker 0: Sort Method: external merge Disk: 34800kB
Worker 1: Sort Method: external merge Disk: 34912kB
-> Parallel Seq Scan on ticket_flights (cost=0.00..29554.73 rows=983473 width=32) (actual time=1.237..1503.942 rows=78
Buffers: shared read=19720
Planning:
Buffers: shared hit=119 read=5 dirtied=5
Planning Time: 11.557 ms
Execution Time: 5021.231 ms
Значение
Gather Merge (cost=151977.82..381470
16 строк получено - 5,049s, 2026-10-31 в 20:20:54
GMT+04:00
Запись
Инт. вставка
2 : 78 : 100


DBeaver 26.2.0 - <demo (1)> Script-22
SELECT pid, state, wait_event_type, wait_event, left(query, 80) AS q
FROM pg_stat_activity
WHERE state = 'active' AND pid <> pg_backend_pid();
pg_stat_activity 1
pid
state
wait_event_type
wait_event
q
1 590 active IO DataFileRead EXPLAIN (ANALYZE, BUFFERS) SELECT * FRC
2 600 active [NULL] [NULL] EXPLAIN (ANALYZE, BUFFERS) SELECT * FRC
3 601 active IO DataFileRead EXPLAIN (ANALYZE, BUFFERS) SELECT * FRC
Значение
590
GMT+04:00
Запись
Инт. вставка
3 : 52 : 144





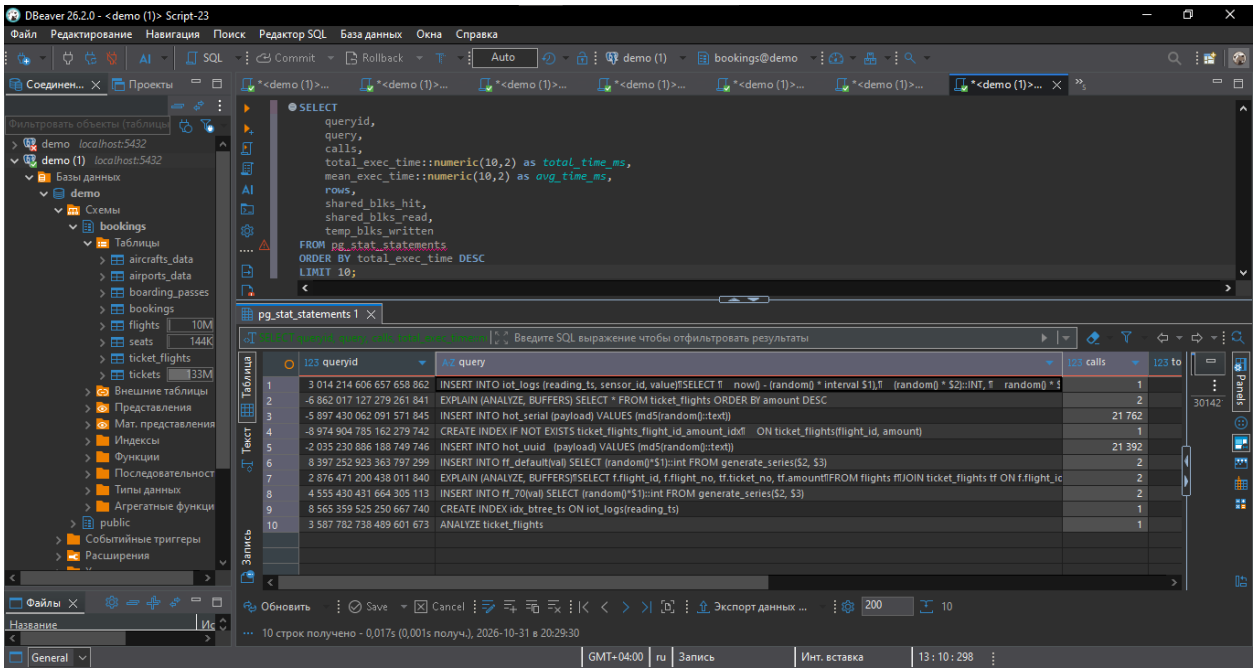
При work_mem = '1MB' в плане видно Sort Method: external merge Disk: 33008kB. В pg_stat_activity в этот момент фиксируется wait_event_type = IO, wait_event = DataFileRead. Это означает, что процессу не хватило памяти, и он ждет чтения/записи временных файлов с диска. Время выполнения –5021 мс.

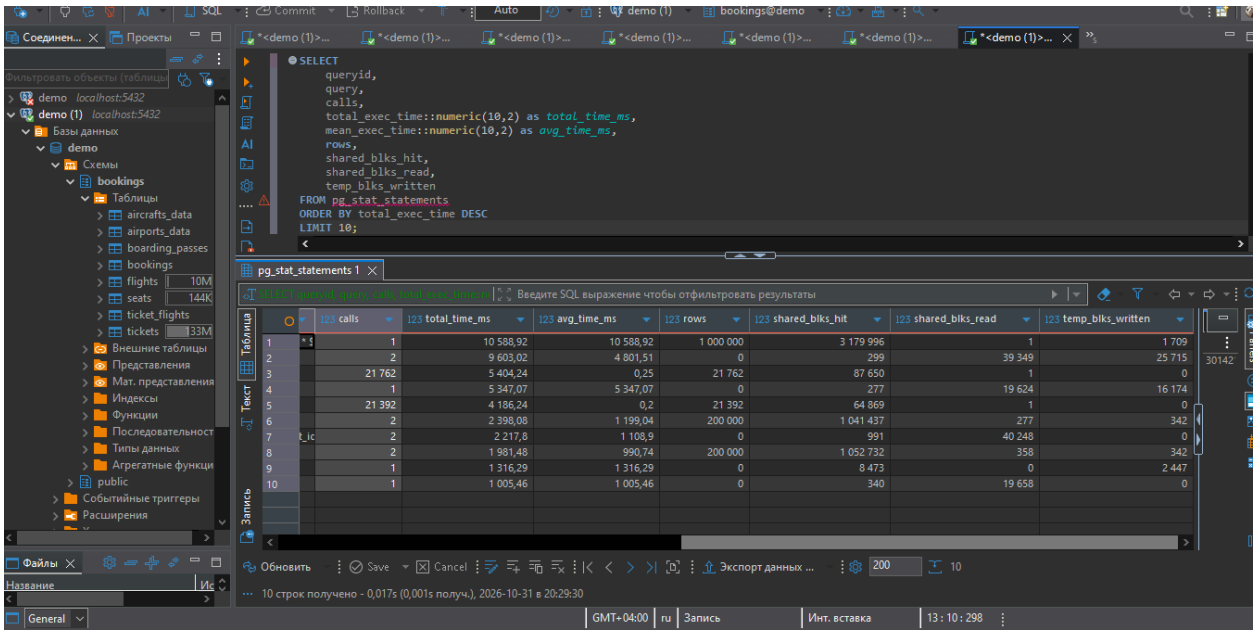
При work_mem = '256MB' в плане Sort Method: quicksort Memory: 219015kB.
В pg_stat_activity ожидания отсутствуют (NULL), так как сортировка полностью идет в оперативной памяти, и процессор не ждет диск. Время выполнения сокращается до 4565 мс.

Вывод: wait_event напрямую отражает Sort Method. Если сортировка уходит на диск (external merge), мы видим ожидание IO/DataFileRead. Если сортировка в памяти, ожиданий ввода-вывода нет.

Раздел 9. Диагностика через pg_stat_statements

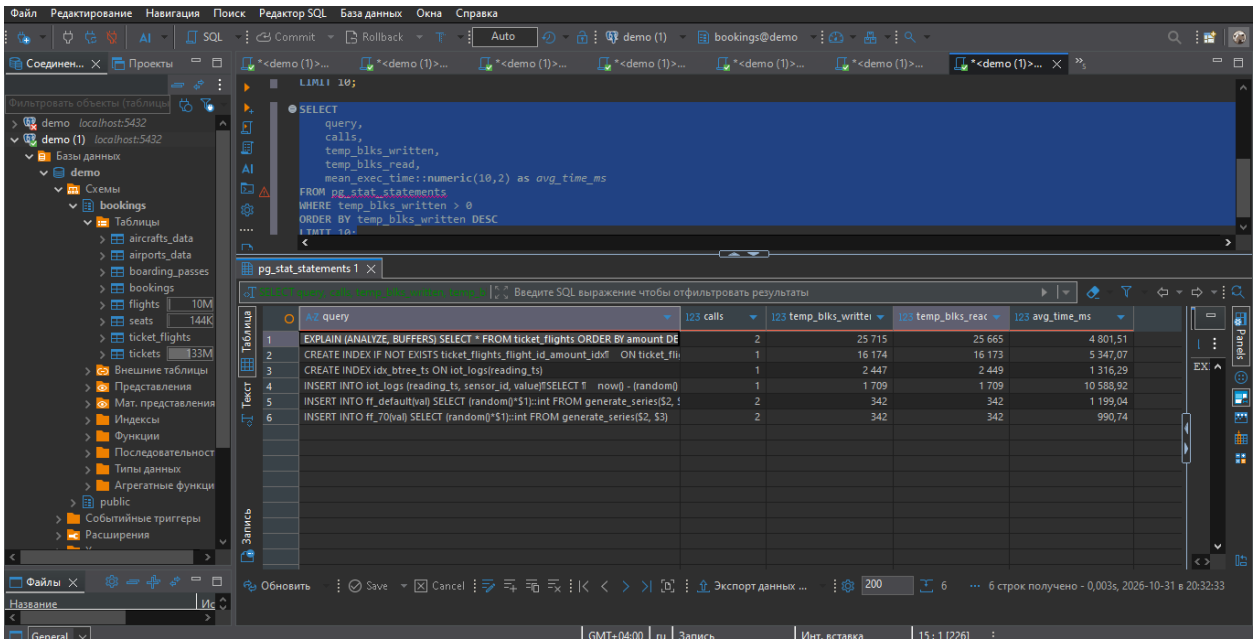
Запрос 1. Топ-10 самых медленных





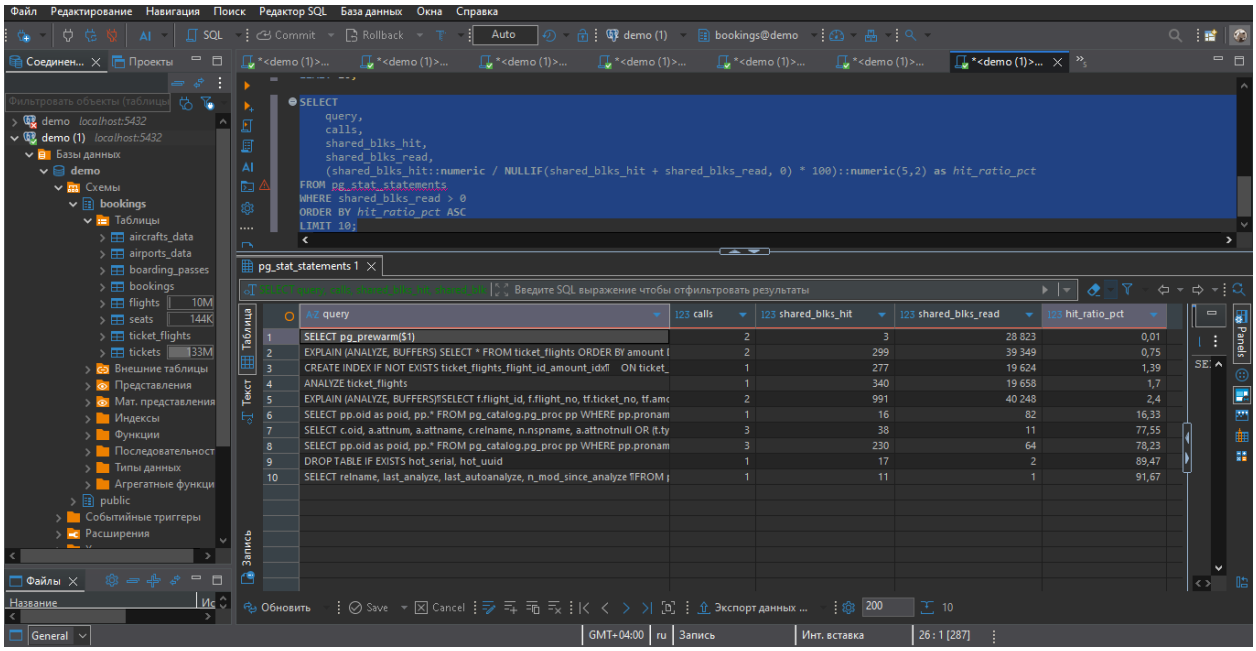
В топ-10 по общему времени выполнения попали запросы из предыдущих заданий: вставка 1 млн строк в iot_logs (5.4 сек), аналитические запросы с JOIN и сортировкой (2.2 сек), а также создание индексов. Это подтверждает, что именно они создают основную нагрузку.

Запрос 2. Запросы со сбросом на диск



Запросы с temp_blks_written > 0 –это те, что уходили в external merge Disk (сортировка с маленьким work_mem), а также CREATE INDEX (который тоже использует временные файлы). Сбросы на диск сильно замедляют работу

Запрос 3. Запросы с низким cache hit ratio



Запросы с самым низким cache hit ratio (0.01%–2.4%) –это те, которые читают данные напрямую с диска (холодный кэш). Например, запросы к системному каталогу pg_catalog (они всегда читают с диска, так как редко кэшируются) и тяжелые аналитические запросы, которые не помещаются в shared_buffers. Высокий hit ratio (90%+) у мелких запросов, которые уже в кэше.

Вывод: pg_stat_statements позволяет увидеть не только время, но и характер нагрузки (чтение с диска, запись во временные файлы). Это главный инструмент для поиска узких мест.

Раздел 7. Теоретическое задание: назначение системных представлений статистики

В данном разделе описаны системные представления и расширения, которые использовались при выполнении заданий.

Объект	Что это	Ключевые столбцы	Что показывает значение	Для чего нужен	Пример из ДЗ	Ограничения
pg_stat_user_tables	Системное представление (view) в схеме pg_catalog.	last_analyze, last_autoanalyze, n_mod_since_analyze, seq_scan, idx_scan, n_live_tup, n_dead_tup.	Счётчики накапливаются с момента сброса статистики. Время анализа – временная метка.	Диагностика устаревания статистики, поиск таблиц без индексов, поиск раздувания (n_dead_tup).	В Задании 1 проверяла last_autoanalyze и n_mod_since_analyze для flights.	Показывает только текущий срез. Счётчики могут сбрасываться при перезапуске сервера.
pg_stat_activity	Системное представление (view) в схеме pg_catalog.	state, wait_event_type, wait_event, query, query_start.	Живой срез текущей активности. Не хранит историю.	Поиск «зависших» запросов, диагностика блокировок, понимание, на что тратится время (I/O или CPU).	В Задании 5 и Разделе 8 смотрела wait_event (DataFileRead) из-за нехватки work_mem.	Видно только то, что происходит сейчас. Поле query обрезается до 1 КБ.
pg_stat_statements	Расширение (extension). Требует CREATE EXTENSION и настройки shared_preload_libraries.	queryid, calls, total_exec_time, mean_exec_time, shared_blks_hit/read, temp_blks_written.	Агрегированная статистика по всем выполненным запросам (накапливается).	Поиск самых ресурсоёмких запросов (топ по времени), поиск запросов с записью на диск, оценка hit ratio.	В Задании 5 нашла тяжелый JOIN по total_exec_time и проверила temp_blks_written.	Не видит запросы до установки расширения. Заменяет литералы на плейсхолдеры. Хранит только 5000 запросов.
pg_class	Системный каталог (таблица) в схеме pg_catalog.	relname, relkind, relpages, reltuples, oid.	Актуальные метаданные (зависят от последнего VACUUM/ANALYZE).	Получение структуры БД, поиск объектов по имени, вычисление размеров через pg_relation_size(oid).	В Задании 4 и Разделе 4 использовала pg_relation_size(oid) для сравнения B-Tree и BRIN.	relpages и reltuples –это оценки, они могут быть неточными для таблиц, куда только что вставили данные.
pg_prewarm	Расширение (extension). Основная функция – pg_prewarm(regclass).	Принимает имя таблицы или индекса (regclass), возвращает число страниц.	Возвращает число страниц, загруженных в shared_buffers.	Принудительный прогрев кэша (чтения в RAM) перед важными запросами, чтобы избежать I/O Stall.	В Задании 2 и Разделе 3 прогревала ticket_flights, чтобы снизить shared read и поднять shared hit.	Не гарантирует, что страницы останутся в кэше (могут быть вытеснены). Не прогревает системные каталоги.
pgstattuple / pgstatindex	Расширение pgstattuple, предоставляющее функции анализа таблиц и индексов.	avg_leaf_density, leaf_fragmentation, leaf_pages, index_size, free_percent.	Физическое состояние объекта (насколько плотно упакованы данные).	Диагностика раздувания (bloat), оценка эффективности fillfactor, понимание, сколько места вернёт REINDEX.	В Разделе 7 использовала pgstatindex для сравнения плотности листовых страниц (fillfactor 90 и 70).	Требует много ресурсов (читает все страницы объекта), работает долго и нагружает диск.
pg_stat_bgwriter (или pg_stat_io)	Системное представление (view) в схеме pg_catalog.	checkpoints_timed/req, buffers_backend, buffers_checkpoint, buffers_alloc.	Накопленные счётчики работы с памятью и диском.	Общий взгляд на I/O нагрузку. Например, рост checkpoints_req говорит о том, что max_wal_size мал.	В ДЗ напрямую не использовался, но относится к управлению I/O.	Не показывает, какие именно запросы генерируют эту нагрузку.

g_stat_database	Системное представление (view) в схеме pg_catalog.	blks_hit, blks_read, tup_returned/fetched, xact_commit/rollback.	Накопленная статистика по всей базе целиком.	Быстрая оценка «здоровья» базы (высокий xact_rollback – проблема в приложении, низкий blks_hit –мало shared_buffers).	Позволяет оценить общий hit ratio всей базы.	Показывает только суммарные цифры, без разбивки по конкретным таблицам или запросам.

Вывод по разделу 7

Наиболее полезными в моей работе
оказались pg_stat_activity и pg_stat_statements. pg_stat_activity позволяет в реальном времени увидеть, чего именно ждет запрос (wait_event), а pg_stat_statements –найти главных «виновников» по времени выполнения. Также pgstatindex очень помог для наглядного понимания того, как fillfactor влияет на физическую структуру.

При следующей диагностике я бы в первую очередь проверяла pg_stat_statements на предмет запросов с высоким total_exec_time и temp_blks_written, а затем через pg_stat_activity смотрела на wait_event, чтобы понять, упирается ли проблема в CPU, блокировки или диск.